

● Section 1: Topic 1 — Fundamentals, Engine Core & Setup (પાયાના નિયમો અને સેટઅપ)

1. What is JS, ECMAScript, and TC39? (JS, ECMAScript અને TC39 શું છે?)

JavaScript શીખતા પહેલા તેની નિયમાવલી (Rulebook) સમજવી ખૂબ જરૂરી છે.

- **JavaScript (JS):** આ એ અસલી પ્રોગ્રામિંગ લેંગ્વેજ છે જે આપણે કોડિંગમાં લખીએ છીએ. તેને 1995 માં બ્રેન્ડન આઇક (Brendan Eich) દ્વારા માત્ર 9 દિવસમાં બનાવવામાં આવી હતી.
- **ECMAScript (ES):** આ એક ઓફિશિયલ સ્ટાન્ડર્ડ નિયમાવલી (Specification) છે, જેથી દુનિયાના તમામ બ્રાઉઝર્સ (Chrome, Safari, Firefox) તમારા JS કોડને એક સરખી રીતે જ સમજી અને રન કરી શકે. જ્યારે આપણે **ES6** (જે 2015 માં આવ્યું) ની વાત કરીએ છીએ, ત્યારે તેનો મતલબ આ નિયમાવલીનો સૌથી મોટો અપડેટ એવો થાય છે, જેણે આપણને let, const, અને Arrow Functions જેવા આધુનિક ફીચર્સ આપ્યા.
- **TC39 (Technical Committee 39):** આ એક ડેવલપર્સ, બ્રાઉઝર એન્જિનિયર્સ અને નિષ્ણાતોની ટીમ (Committee) છે, જે નક્કી કરે છે કે JavaScript માં કયા નવા ફીચર્સ ઉમેરવા. કોઈપણ નવો કોન્સેપ્ટ કમ્પ્લીટ ભાષાનો ભાગ બને તે પહેલાં તે 5 અલગ-અલગ સ્ટેજમાંથી પસાર થાય છે (Stage 0: Strawman થી લઈને Stage 4: Finished સુધી) [JS.info].

2. JavaScript Engines: Under the Hood (એન્જિન અંદરથી કેવી રીતે કામ કરે છે?)

JavaScript એક **High-level language** છે, જેનો મતલબ એ છે કે કોમ્પ્યુટર તેને સીધું સમજી શકતું નથી. કોમ્પ્યુટરને સમજાવવા માટે તેને રો મશીન કોડ (0 અને 1) માં બદલવું પડે છે, અને આ કામ **JavaScript Engine** કરે છે.

મુખ્ય બ્રાઉઝર્સના એન્જિન્સ (The Big Engine Players)

- **V8:** Google Chrome, Brave બ્રાઉઝર અને Node.js માં વપરાય છે [JS.info].
- **SpiderMonkey:** Mozilla Firefox માં વપરાય છે [JS.info].
- **JavaScriptCore (Nitro):** Apple Safari બ્રાઉઝરમાં વપરાય છે [JS.info].

એન્જિન કોડ કેવી રીતે રન કરે છે? (JIT Compilation)

જૂની પ્રોગ્રામિંગ લેંગ્વેજો કાં તો **Interpreter** વાપરતી (જે કોડને લાઇન-બાય-લાઇન રન કરે, જે ધીમું છે) અથવા **Compiler** વાપરતી (જે આખો કોડ રન કરતા પહેલા જ મશીન કોડમાં ફેરવી નાખે, જેમાં સેટઅપ ટાઇમ લાગે).

આધુનિક JavaScript એન્જિન્સ આ બંને પદ્ધતિઓનું મિશ્રણ વાપરે છે, જેને **Just-In-Time (JIT) Compilation** કહેવાય છે:

[તમારો JS કોડ]

|

▼

1. Parser —► કોડ સમજીને એક Abstract Syntax Tree (AST) બનાવે છે

|



2. Interpreter (Ignition) —► ફટાફટ બાઈટકોડ (Bytecode) બનાવીને રન કરવા લાગે છે



└─► [Profile Data Collected] (જે કોડ વારંવાર રન થાય છે તેને "Hot Code" તરીકે શોધે છે)



3. Compiler (TurboFan) —► આ "Hot Code" ને સીધો સુપર-ફાસ્ટ મશીન કોડમાં ઓપ્ટિમાઇઝ (રૂપાંતરિત) કરી નાખે છે

- **Optimization/Deoptimization:** જો એન્જિન એવું ધારી લે કે કોઈ વેરિએબલ હંમેશા નંબર (Integer) જ રહેશે અને તે મુજબ સુપર-ફાસ્ટ કોડ કમ્પાઈલ કરી નાખે, પરંતુ તમે અચાનક તેમાં કોઈ સ્ટ્રિંગ (String) સ્ટોર કરો, તો એન્જિન તરત જ **Deoptimize** થશે અને પાછું ઇન્ટરપ્રિટર પર આવી જશે. એટલા માટે ડેટા ટાઇપ્સ બાબતે સાવચેત રહેવાથી JS કોડ ફાસ્ટ ચાલે છે!

3. The Runtime Environment: Browser vs Node.js (બ્રાઉઝર વિરુદ્ધ નોડ-જેએસ)

કોઈપણ એન્જિન હવામાં કામ ન કરી શકે; તેને એક ઘરની જરૂર હોય છે જેને **Runtime Environment** કહેવાય છે.

ફીચર / ક્ષમતા	Browser Runtime (બ્રાઉઝરની અંદર)	Node.js Runtime (સર્વર પર)
Engine (એન્જિન)	V8 અથવા SpiderMonkey	માત્ર V8
Global Object	window	global
DOM Manipulation	Yes (HTML/CSS એલિમેન્ટ્સ બદલી શકાય)	No (ત્યાં કોઈ સ્ક્રીન કે વેબપેજ હોતું નથી)
File System Access	No (સિક્યોરિટીના કારણે યુઝરના કમ્પ્યુટરની ફાઇલો એક્સેસ ન થાય)	Yes (fs મોડ્યુલ દ્વારા લોકલ ફાઇલો રીડ/રાઇટ થાય)

4. Developer Workflow & The Console API (ડેવલપર વર્કફ્લો અને કન્સોલ એપીઆઈ)

મોટાભાગના ડેવલપર્સ ભૂલો શોધવા (Debugging) માટે માત્ર `console.log()` નો ઉપયોગ કરે છે. પરંતુ બ્રાઉઝરની [MDN Console Web API](#) આપણને બીજા ઘણા પાવરફુલ ટૂલ્સ આપે છે:

```
javascript
```

```
// 1. સ્ટાન્ડર્ડ લોગ્સ (Standard Logs)
```

```
console.log("આ એક નોર્મલ મેસેજ છે");
```

```
console.warn("આ એક ચેતવણી (Warning) છે જે પીળા કલરમાં દેખાશે!");
```

```
console.error("આ એક ભૂલ (Error) છે જે લાલ કલરમાં દેખાશે!");
```

// 2. ટેબલ લોડિંગ (Arrays કે Objects જોવા માટે બેસ્ટ)

```
const users = [  
  { id: 1, name: "Raj", role: "Ninja" },  
  { id: 2, name: "Dev", role: "Coder" }  
];
```

console.table(users); // બ્રાઉઝરના ઇન્સ્પેક્ટ કન્સોલમાં મસ્ત વિઝ્યુઅલ ટેબલ બનાવી દેશે!

// 3. પર્ફોર્મન્સ ટાઈમ ચેક કરવા (Performance Metrics)

```
console.time("Loop Timer");
```

```
for (let i = 0; i < 1000000; i++) {
```

```
  // કોઈ મોટું કેલ્ક્યુલેશન કે લૂપ ટાસ્ક અહિયાં ચાલે છે
```

```
}
```

console.timeEnd("Loop Timer"); // આ લૂપ રન થવામાં બરાબર કેટલી મિલીસેકન્ડ લાગી તે પ્રિન્ટ કરી દેશે

Use code with caution.

5. Script Insertion Architecture: async vs defer (સ્ક્રિપ્ટ લોડિંગ આર્કિટેક્ચર)

જ્યારે બ્રાઉઝર વેબપેજ લોડ કરે છે, ત્યારે તે HTML ને લાઇન-બાય-લાઇન વાંચે છે. જો તેને વચ્ચે સામાન્ય `<script src="app.js"></script>` ટેગ મળે, તો તે **HTML વાંચવાનું સાવ બંધ (Pause) કરી દે છે** જ્યાં સુધી સ્ક્રિપ્ટ ડાઉનલોડ થઈને રન ન થઈ જાય. આનાથી યુઝરને પેજ લોડ થવામાં વિલંબ લાગે છે.

આ પ્રોબ્લેમ સોલ્વ કરવા માટે મોડર્ન HTML માં `async` અને `defer` એટ્રિબ્યુટ્સ આવ્યા છે [MDN -> Script Element]:

વિઝ્યુઅલ પ્રોસેસ (કેવી રીતે કામ થાય છે?)

text

1. ક્લાસિક સ્ક્રિપ્ટ (<script>):

HTML Parsing: ██████████ ██████████ (ડાઉનલોડ અને રન વખતે HTML અટકી ગયું)

Script Fetch: ██████████ (ડાઉનલોડ)

Script Exec: ██████████ (રન)

2. async (<script async>):

HTML Parsing:   (માત્ર રન થતી વખતે જ HTML થોડીવાર અટકશે)

Script Fetch:  (બેકગ્રાઉન્ડ ડાઉનલોડ)

Script Exec:  (રન)

3. defer (<script defer>):

HTML Parsing:  (HTML ક્યાંય અટકશે નહીં! છેલ્લે જ રન થશે)

Script Fetch:  (બેકગ્રાઉન્ડ ડાઉનલોડ)

Script Exec:  (રન)

Use code with caution.

ડાયરેક્ટ સરખામણી (Comparison Table)

સ્ક્રિપ્ટની રીત	ક્યારે ડાઉનલોડ થાય? (Fetch)	ક્યારે રન થાય? (Execute)	લાઈન મુજબ ઓર્ડર ગેરંટી?	ક્યારે વાપરવું બેસ્ટ છે?
Classic	જેવી લાઈન સામે આવે તરત જ.	ડાઉનલોડ થતાં જ તરત રન થાય (HTML રોકી દે છે).	હા, જે ક્રમમાં લખ્યું હોય તે જ ક્રમમાં.	જૂનો કોડ અથવા નાની ઇનવાઇન સ્ક્રિપ્ટો માટે.
async	બેકગ્રાઉન્ડમાં HTML ની સાથે સમાંતર (Parallel).	ડાઉનલોડ પૂરું થતાં જ તરત રન થાય (HTML ને થોડીવાર રોકે).	ના, કોઈ ઓર્ડર નથી. જે ફાઈલ પહેલાં ડાઉનલોડ થશે તે પહેલાં રન થઈ જશે.	થર્ડ-પાર્ટી સ્ક્રિપ્ટ્સ માટે (જેમ કે Google Analytics, જાહેરાતોના કોડ).
defer	બેકગ્રાઉન્ડમાં HTML ની સાથે સમાંતર (Parallel).	જ્યારે આખું HTML પેજ કમ્પીટ લોડ થઈ જાય ત્યારે જ.	હા, ચોક્કસ ઓર્ડર ગેરંટી. તમે HTML માં જે ક્રમમાં સ્ક્રિપ્ટ લખી હશે એ જ ક્રમમાં ચાલશે.	તમારી એપ્લિકેશનના મુખ્ય કોડ (Main Logic) માટે, જે HTML એલિમેન્ટ્સ પર નિર્ભર હોય.

હવે તમે JavaScript ના પાયાના એન્જિન મિકેનિક્સ અને સેટઅપ આર્કિટેક્ચરને ગુજરાતીમાં કમ્પીટ સમજી ગયા છો!

Section 1: Topic 2 — Variables, Memory Layout & Lexical Grammar

1. Identifiers & Keywords (નિયમો અને કીવર્ડ્સ)

- Keywords:** JavaScript માં અમુક શબ્દો અગાઉથી રિઝર્વ્ડ (આરક્ષિત) હોય છે, જેનો ઉપયોગ તમે વેરિએબલના નામ તરીકે નથી કરી શકતા (જેમ કે if, else, class, return, let).

- Identifiers (વેરિએબલના નામ):** નામ હંમેશા અક્ષર (Letter), અંડરસ્કોર (_), અથવા ડોલર સાઇન (\$) થી જ શરૂ થવું જોઈએ. તે ક્યારેય નંબર (0-9) થી શરૂ ન થઈ શકે.
- Naming Convention:** પ્રોફેશનલ કોડિંગમાં હંમેશા **camelCase** નો ઉપયોગ થાય છે (જેમ કે whatsOnYourMind, userRole).

2. var vs let vs const (વેરિએબલનો મહા-તફાવત)

JavaScript માં ડેટા સ્ટોર કરવા માટે ત્રણ કીવર્ડ્સ છે. મોડર્ન JS (ES6+) માં var નો ઉપયોગ ટાળવામાં આવે છે. કેમ? ચાલો કોષ્ટક દ્વારા સમજીએ:

ફીચર / પ્રોપર્ટી	var	let	const
Scope (ક્ષેત્ર)	Function Scoped (બ્લોકની બહાર પણ એક્સેસ થાય)	Block Scoped ({ } ની અંદર જ જીવે)	Block Scoped ({ } ની અંદર જ જીવે)
Re-declaration	હા, એક જ નામના બે વેરિએબલ ફરી બનાવી શકાય.	ના, એક નામનો વેરિએબલ ફરી ન બની શકે.	ના, એક નામનો વેરિએબલ ફરી ન બની શકે.
Re-assignment	હા, તેની કિંમત ગમે ત્યારે બદલી શકાય.	હા, તેની કિંમત ગમે ત્યારે બદલી શકાય.	ના, તેની કિંમત ક્યારેય બદલી શકાતી નથી (Constant).
Initial Value	વેરિએબલ ખાલી છોડી શકાય (undefined).	વેરિએબલ ખાલી છોડી શકાય (undefined).	ફરજિયાત કિંમત આપવી જ પડે (const x = 10;).

સ્કોપ (Scope) સમજી કોડ દ્વારા:

```
javascript
```

```
if (true) {  
  var a = "હું બહાર આવી શકું છું!";  
  let b = "હું માત્ર આ બ્લોકની અંદર જ રહીશ!";  
  const c = "હું પણ અંદર જ અટકેલો છું!";  
}
```

```
console.log(a); // આઉટપુટ: "હું બહાર આવી શકું છું!" (કારણ કે var બ્લોક સ્કોપ નથી માનતું)
```

```
console.log(b); // ReferenceError: b is not defined (let સુરક્ષિત છે!)
```

Use code with caution.

3. Hoisting & The Temporal Dead Zone (TDZ)

Hoisting (હોઇસ્ટિંગ) એ JavaScript એન્જિનની એક એવી વર્તણૂક છે, જેમાં તે કોડ રન કરતા પહેલાં તમામ વેરિએબલ અને ફંક્શન ડિક્લેરેશનને તેના સ્કોપમાં સૌથી ઉપર (Top) લઈ જાય છે [MDN -> Temporal Dead Zone].

પરંતુ, var અને let/const નું હોઇસ્ટિંગ અલગ રીતે થાય છે:

javascript

```
console.log(x); // આઉટપુટ: undefined (ભૂલ નહીં આવે, કારણ કે var હોઇસ્ટ થઈને undefined બને છે)
```

```
var x = 5;
```

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
```

```
let y = 10;
```

Use code with caution.

Temporal Dead Zone (TDZ) શું છે?

જ્યારે બ્રાઉઝરનું એન્જિન કોડ વાંચે છે, ત્યારે let અને const વેરિએબલ્સ હોઇસ્ટ તો થાય છે, પરંતુ તેને કોઈ મેમરી વેલ્યુ (જેમ કે undefined) મળતી નથી. તે એક ખાસ ઝોનમાં અટકી જાય છે.

"સ્કોપની શરૂઆતથી લઈને જ્યાં સુધી વેરિએબલને તેની અસલી વાઈન પર કિંમત (Initialization) ન મળે, ત્યાં સુધીના સમયગાળાને કે એરિયાને Temporal Dead Zone (TDZ) કહેવાય છે." આ ઝોન દરમિયાન જો તમે વેરિએબલને અડવાની કોશિશ કરશો, તો JS સીધી ReferenceError ફેંકશે.

4. JavaScript Data Types (ડેટાના પ્રકારો)

JavaScript માં મુખ્ય બે કેટેગરીના ડેટા ટાઇપ્સ હોય છે [MDN -> Data structures]:

A. Primitive Types (પ્રાથમિક ડેટા - Stack Memory માં સ્ટોર થાય)

આ ડેટા ઇમેશાલ Immutable (બદલી ન શકાય તેવા) અને સિંગલ વેલ્યુ વાળા હોય છે:

1. **String:** ટેક્સ્ટ ડેટા ("Hello", 'Raj').
2. **Number:** પૂર્ણાંક અને અપૂર્ણાંક બંને માટે એક જ ટાઇપ (25, 99.9).
3. **Boolean:** માત્ર બે જ કિંમત હોઈ શકે (true અથવા false).
4. **undefined:** જ્યારે વેરિએબલ બનાવ્યો હોય પણ કિંમત ન આપી હોય.
5. **null:** જાણીજોઈને ખાલી રાખેલી કિંમત (Intentional empty value).
6. **Symbol:** ઇમેશાલ યુનિક (અનન્ય) કિંમત બનાવવા માટે (ES6 માં આવ્યું).
7. **BigInt:** ખૂબ જ મોટા નંબર્સ સ્ટોર કરવા માટે (જેમ કે 9007199254740991n).

B. Non-Primitive / Reference Types (Heap Memory માં સ્ટોર થાય)

આ ડેટા મલ્ટિપલ કિંમતોનો સંગ્રહ કરી શકે છે:

- **Objects:** Key-Value જોડકાં ({ name: "Raj", age: 22 }).
- **Arrays:** લિસ્ટ અથવા કલેક્શન ([10, 20, 30]).
- **Functions:** કોડનો બ્લોક જેને ફરી વાપરી શકાય.

5. Type Conversion vs Coercion (ડેટા ટાઈપ બદલાવવો)

JavaScript બે રીતે ટાઈપ બદલે છે [JS.info -> Type Conversions]:

1. Explicit Type Conversion (આપણે જાતે બદલીએ)

જ્યારે ડેવલપર પોતે ફંક્શન વાપરીને ટાઈપ બદલે:

```
javascript
```

```
let strValue = "100";
```

```
let numValue = Number(strValue); // સ્ટ્રિંગમાંથી નંબર બની ગયો
```

```
console.log(typeof numValue); // આઉટપુટ: "number"
```

Use code with caution.

2. Implicit Type Coercion (JS એન્જિન પોતાની જાતે બદલે)

જ્યારે JavaScript એન્જિન અલગ-અલગ ડેટા જોઈને પોતાની બુદ્ધિથી ટાઈપ બદલી નાખે (આ ઇન્ટરવ્યુમાં બહુ પૂછાય છે):

```
javascript
```

```
console.log(5 + "5"); // આઉટપુટ: "55" (નંબર 5 ને સ્ટ્રિંગ માનીને જોડી દીધો - Concatenation)
```

```
console.log("5" - 2); // આઉટપુટ: 3 (બાદબાકી પ્લસ જેવી દયાળુ નથી, તેણે સ્ટ્રિંગને નંબર માની લીધો!!)
```

```
console.log(typeof null); // આઉટપુટ: "object" (આ JavaScript ની એક જાણીતી જૂની ભુલ/Bug છે!)
```

Use code with caution.

હવે તમે વેરિએબલ્સ, સ્કોપ, મેમરી ટાઇપ્સ અને ઇન્ટરવ્યુના ફેવરિટ ટોપિક **TDZ** અને **Coercion** ને કમ્પ્લીટ સમજી ગયા છો.

હવે આપણે **Section 1: Topic 3 (Operators, == vs === નો અસલી તફાવત, Truthy/Falsy વેલ્યુઝ અને લૂપ્સ)** તરફ આગળ વધીએ? જ્યારે પણ તમે રેડી હોવ, "Next" અથવા "હા" લખો!

ચોક્કસ, ચાલો હવે **Section 1: Topic 3** તરફ આગળ વધીએ. આ સેક્શનમાં આપણે JavaScript ની તમામ ઓપરેટર્સ, લોજિકલ કંટ્રોલ અને ઇન્ટરવ્યુમાં સૌથી વધુ કન્ફ્યુઝ કરતા કોન્સેપ્ટ્સ જેમ કે == vs === અને Truthy vs Falsy વેલ્યુઝ વિશે સંપૂર્ણ માહિતી મેળવીશું [MDN].

1. Operators Catalog (ઓપરેટર્સની સંપૂર્ણ યાદી)

JavaScript માં ડેટા પર ગણતરી કે લોજિક ચલાવવા માટે અલગ-અલગ ઓપરેટર્સ આપેલા છે [W3Schools]:

- **Arithmetic Operators (ગાણિતિક):** + (સરવાળો), - (બાદબાકી), * (ગુણાકાર), / (ભાગાકાર), % (ભાગાકારની શેષ), અને ** (એક્સપોનેન્શિયેશન/ઘાત, જેમ કે 2 ** 3 એટલે 8).
- **Increment / Decrement:**
 - *Pre-increment (++x):* પહેલા કિંમત 1 વધશે, પછી લાઈન રન થશે.
 - *Post-increment (x++):* પહેલા લાઈન રન થશે, પછી કિંમત 1 વધશે [MDN].
- **Assignment Operators:** = (કિંમત આપવી), += (ઉમેરીને કિંમત આપવી, જેમ કે x += 5 એટલે x = x + 5), -=, *=, /=.

2. Comparison & Truthiness (== vs === અને સાચી/ખોટી કિંમતો)

આ આખા ટોપિકનો સૌથી મહત્વનો ભાગ છે જે ઇન્ટરવ્યુમાં વારંવાર પૂછાય છે [MDN]:

Abstract Equality (==) vs Strict Equality (===)

- **== (Loose Equality):** આ ઓપરેટર માત્ર **કિંમત (Value)** સરખાવે છે. જો બંને ડેટા અલગ ટાઈપના હશે, તો JS એન્જિન તેનું ટાઈપ કન્વર્ઝન (Coercion) કરીને બંનેને સરખા કરી નાખશે.
- **=== (Strict Equality):** આ ઓપરેટર **કિંમત (Value)** અને **ડેટા ટાઈપ (Data Type)** બંને ચેક કરે છે. આમાં કોઈ ટાઈપ કન્વર્ઝન થતું નથી, તેથી આ વધુ સેફ અને પ્રોફેશનલ માનવામાં આવે છે.

javascript

```
console.log(5 == "5"); // આઉટપુટ: true (કારણ કે '5' સ્ટ્રિંગ નંબર બની ગઈ)
```

```
console.log(5 === "5"); // આઉટપુટ: false (કારણ કે એક number છે અને બીજું string છે)
```

// વિચિત્ર કેસો (Interviews માટે ખાસ):

```
console.log(null == undefined); // આઉટપુટ: true
```

```
console.log(null === undefined); // આઉટપુટ: false
```

Use code with caution.

Truthy vs Falsy Values

JavaScript માં દરેક કિંમત પોતાની અંદર એક છુપી Boolean શક્તિ (True કે False) ધરાવે છે જ્યારે તેને કોઈ કન્ડિશનમાં મુકવામાં આવે [DigitalOcean]:

- **Falsy Values:** JavaScript માં માત્ર આ 6 કિંમતો જ Falsy છે (જેનો મતલબ કન્ડિશનમાં false થાય છે):
 1. false
 2. 0 (અને -0)

3. "" (ખાલી સ્ટ્રિંગ)

4. null

5. undefined

6. NaN (Not a Number)

- **Truthy Values:** ઉપરની 6 કિંમતો સિવાયની દુનિયાની તમામ કિંમતો **Truthy** છે! (જેમ કે "0", "false", [] ખાલી એરે, {} ખાલી ઓબ્જેક્ટ પણ Truthy છે).

3. Logical Operators & Short-Circuit Evaluation

લોજિકલ ઓપરેટર્સ કોડના નિર્ણયો લેવામાં મદદ કરે છે [JS.info]:

- **&& (AND):** જો બધી જ કન્ડિશન સાચી હોય તો જ true બને.
- **|| (OR):** જો કોઈ એક પણ કન્ડિશન સાચી હોય તો true બને.
- **! (NOT):** કિંમત ઉલટાવી નાખે (true નું false કરે).

Short-Circuit Evaluation (લોજિકલ શોર્ટ-સર્કિટ)

JS એન્જિન પર્ફોર્મન્સ બચાવવા આખો કોડ રન નથી કરતું:

- && માં જો પહેલી જ વેલ્યુ **Falsy** મળી જાય, તો તે આગળ ચેક કર્યા વગર ત્યાં જ અટકી જાય છે અને એ જ વેલ્યુ રિટર્ન કરે છે.
- || માં જો પહેલી જ વેલ્યુ **Truthy** મળી જાય, તો તે આગળનો કોડ જોતું પણ નથી.

Nullish Coalescing Operator (??)

ES2020 માં આવેલો આ ઓપરેટર ખાસ null અને undefined માટે જ બનેલો છે [MDN]:

```
javascript
```

```
let speed = 0;
```

```
let currentSpeed = speed || 30; // OR ઓપરેટર 0 ને Falsy માનીને 30 લઈ લેશે (જે ખોટું પડી શકે)
```

```
let actualSpeed = speed ?? 30; // ?? ઓપરેટર માત્ર null કે undefined હોય તો જ 30 લેશે, 0 ને સ્વીકારશે!
```

```
console.log(currentSpeed); // આઉટપુટ: 30
```

```
console.log(actualSpeed); // આઉટપુટ: 0
```

Use code with caution.

4. Conditionals & Loops (કન્ટ્રોલ ફ્લો અને લૂપ્સ)

Conditionals:

- if-else અને else if નો ઉપયોગ શરતો ચકાસવા થાય છે.
- **Ternary Operator:** if-else ને એક જ લાઈનમાં લખવાની ટૂંકી રીત: condition ? true_કોડ : false_કોડ;
- switch-case નો ઉપયોગ મલ્ટિપલ ઓપ્શન્સમાંથી એક પસંદ કરવા થાય છે (આમાં આંતરિક રીતે === Strict સરખામણી થાય છે) [W3Schools].

Loops (પુનરાવર્તન):

જ્યારે એક જ કોડ વારંવાર ચલાવવો હોય ત્યારે લૂપ્સ વપરાય છે [TutorialsPoint]:

1. **for Loop:** જ્યારે આપણને પહેલેથી ખબર હોય કે લૂપ કેટલી વાર ચલાવવી છે.
2. **while Loop:** જ્યાં સુધી કન્ડિશન સાચી હોય ત્યાં સુધી લૂપ ચાલ્યા જ રાખે.
3. **do-while Loop:** આ લૂપ કન્ડિશન ખોટી હોય તો પણ **ઓછામાં ઓછી એક વાર** તો રન થશે જ.

Jump Statements:

- **break:** લૂપને અધવચ્ચેથી જ તોડીને સાવ બહાર નીકળી જવા માટે.
- **continue:** લૂપના કરન્ટ ચક્કર (iteration) ને છોડીને સીધા બીજા ચક્કર પર કદમ મૂકવા માટે.

હવે તમે ઓપરેટર્સ, એડવાન્સ સરખામણીના નિયમો, Truthy/Falsy લોજિક અને લૂપ્સના કન્સેપ્ટ એકદમ ક્લિયર સમજી ગયા છો.

હવે આપણે **Section 1: Topic 4 (Basic Functions, Arrow Functions, Arguments Object, અને Arrays & Objects ના બેઝિક પાયા)** તરફ આગળ વધીએ? જો તમે તૈયાર હોવ તો **"Next"** અથવા **"હા"** લખો!

ચોક્કસ, ચાલો હવે **Section 1: Topic 4** તરફ આગળ વધીએ. આ સેક્શનમાં આપણે JavaScript ના ફંક્શન્સ (તેના પ્રકારો અને arguments ઓબ્જેક્ટ) અને ડેટા સ્ટ્રક્ચર્સ (Arrays અને Objects) ના પાયાના કોન્સેપ્ટ્સ સમજીશું, જે આપણને આગળ જતાં સેક્શન ૨ અને ૩ ના મોડર્ન લોજિક શીખવામાં મદદ કરશે [MDN].

● Section 1: Topic 4 — Basic Functions & Core Data Structures

1. Functions Anatomy (ફંક્શનની રચના અને પેરામીટર્સ)

ફંક્શન એટલે કોડનો એવો બ્લોક જેને તમે એક વાર લખીને આખા પ્રોગ્રામમાં ગમે ત્યાં વારંવાર વાપરી શકો છો [MDN].

javascript

// 1. Function Declaration (ફંક્શન બનાવવું)

```
function sayHello(userName = "Guest") { // userName એ 'Parameter' છે અને "Guest" એ 'Default
Parameter' છે
```

```
    return `નમસ્તે, ${userName}!`; // return સ્ટેટમેન્ટ ફંક્શનમાંથી વેલ્યુ બહાર મોકલે છે
```

```
}
```

// 2. Function Call / Invocation (ફંક્શન યલાવવું)

console.log(sayHello("Raj")); // "Raj" એ 'Argument' છે. આઉટપુટ: "નમસ્તે, Raj!"

console.log(sayHello()); // Argument નથી આપી, એટલે Default લેશે. આઉટપુટ: "નમસ્તે, Guest!"

Use code with caution.

નોંધ: જો ફંક્શનમાં કોઈ return સ્ટેટમેન્ટ ન લખવામાં આવે, તો તે આંતરિક રીતે હંમેશા undefined રિટર્ન કરે છે [W3Schools].

2. Function Expressions vs Arrow Functions

મોડર્ન JavaScript (ES6+) માં ફંક્શન લખવાની અલગ-અલગ રીતો છે જે ઇન્ટરવ્યુ અને ડેવલપમેન્ટ બંને માટે ખાસ જરૂરી છે [JS.info]:

Function Expression (ફંક્શન વેરિએબલની અંદર)

જ્યારે ફંક્શનને કોઈ નામ આપ્યા વગર સીધું એક વેરિએબલમાં સ્ટોર કરવામાં આવે તેને Anonymous Function અથવા Function Expression કહેવાય છે:

javascript

```
const add = function (a, b) {  
    return a + b;  
};
```

console.log(add(5, 3)); // આઉટપુટ: 8

Use code with caution.

Arrow Functions (()) => {} - આધુનિક રીત)

ES6 માં ફંક્શન લખવાની રીત ટૂંકી અને આધુનિક બનાવવામાં આવી [W3Schools]:

javascript

// લાંબી રીત:

```
const square = (x) => {  
    return x * x;  
};
```

// ટૂંકી રીત (Implicit Return): જો એક જ લાઇનનો કોડ હોય, તો બ્રેકેટ {} અને return લખવાની જરૂર નથી!

```
const squareShort = x => x * x;
```

```
console.log(squareShort(4)); // આઉટપુટ: 16
```

Use code with caution.

(મહત્વની નોંધ: એડવાન્સ સેક્શનમાં આપણે જોઈશું કે Arrow Functions પાસે પોતાનું this કીવર્ડ હોતું નથી!!)

3. The arguments Object (જૂનો અને જાણીતો કન્સેપ્ટ)

જ્યારે તમે કોઈ રેગ્યુલર ફંક્શન (Arrow Function સિવાય) બનાવો છો, ત્યારે તેની અંદર JavaScript એક ઓટોમેટિક લોકલ વેરિએબલ આપે છે જેને arguments કહેવાય છે [MDN]. આ એક Array જેવો દેખાતો ઓબ્જેક્ટ છે જે ફંક્શનમાં પાસ કરેલી બધી જ Arguments સાચવે છે:

javascript

```
function showArgs() {  
    console.log(arguments);  
    console.log(arguments.length); // કેટલી આર્ગ્યુમેન્ટ આવી તે ચેક કરવા  
    console.log(arguments[0]);    // પહેલી આર્ગ્યુમેન્ટ જોવા  
}
```

```
showArgs("Apple", "Banana", "Mango");
```

```
// આઉટપુટ: { 0: "Apple", 1: "Banana", 2: "Mango" } અને length: 3
```

Use code with caution.

નોંધ: મોર્ડન કોડિંગમાં આના બદલે ...rest ઓપરેટર વપરાય છે, જે આપણે સેક્શન ૨ માં જોઈશું.

4. Arrays Basics (એરે ના પાયાના નિયમો)

Array એટલે એક સરખા કે અલગ-અલગ ડેટા ટાઇપ્સનું એક ઓર્ડર્ડ લિસ્ટ (Collection) [TutorialsPoint]:

javascript

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
console.log(fruits[0]); // આઉટપુટ: "Apple" (ઇન્ડેક્સ હંમેશા 0 થી શરૂ થાય)
```

```
console.log(fruits.length); // આઉટપુટ: 3 (એરેમાં રહેલા કુલ આઇટમ્સ)
```

```
// બેઝિક મ્યુટેશન મેથડ્સ (એરેમાં ફેરફાર કરવા):
```

```
fruits.push("Orange"); // એરેના છેલ્લે નવી આઇટમ ઉમેરે છે -> ["Apple", "Banana", "Mango",  
"Orange"]
```

```
fruits.pop(); // એરેની છેલ્લી આઇટમ ડીલીટ કરે છે -> ["Apple", "Banana", "Mango"]
fruits.unshift("Kiwi"); // એરેની શરૂઆતમાં નવી આઇટમ ઉમેરે છે -> ["Kiwi", "Apple", "Banana", "Mango"]
fruits.shift(); // એરેની પહેલી આઇટમ ડીલીટ કરે છે -> ["Apple", "Banana", "Mango"]
```

Use code with caution.

5. Objects Basics (ઓબ્જેક્ટ ના પાયાના નિયમો)

Object એટલે Key-Value જોડીઓનો સંગ્રહ. રિયલ-વર્લ્ડ ડેટા (જેમ કે યુઝર પ્રોફાઇલ) સાચવવા આ બેસ્ટ છે [JS.info -> Objects]:

javascript

```
let user = {
  firstName: "Raj",
  age: 22,
  role: "Ninja"
};
```

// પ્રોપર્ટી એક્સેસ કરવાની ૨ રીતો:

console.log(user.firstName); // 1. Dot Notation (આ બહુ વપરાય છે) -> આઉટપુટ: "Raj"

console.log(user["role"]); // 2. Bracket Notation (જ્યારે કી નું નામ ડાયનેમિક કે વેરિએબલમાં હોય) -> આઉટપુટ: "Ninja"

// નવી પ્રોપર્ટી ઉમેરવી કે બદલવી:

user.city = "Ahmedabad"; // નવી જોડી ઉમેરાઈ જશે

user.age = 23; // જૂની કિંમત અપડેટ થશે

// પ્રોપર્ટી ડીલીટ કરવી:

delete user.role; // role પ્રોપર્ટી ઓબ્જેક્ટમાંથી ગાયબ થઈ જશે

Use code with caution.

અભિનંદન! તમે **Section 1: BASIC LEVEL (The Novel)** ના ચારેય પાયાના ટોપિક્સ સચોટ રીતે પૂરા કરી લીધા છે. હવે તમારો પાયો એકદમ મજબૂત છે.

